DISCIPLINA · TESTE DE SOFTWARE

Refatoração de Testes e Detecção de Test Smells

Análise, refatoração e verificação estática da suíte de testes do serviço de gerenciamento de usuários (*UserService*) do projeto **test-smelly**.

ALUNO	Enzo Casaes
MATRÍCULA	856804
DISCIPLINA	Teste de Software
PROJETO-BASE	github.com/CleitonSilvaT/test-smelly

CONTEÚDO

1. Análise de três Test Smells da suíte original (`userService.smelly.test.js`)
2. Refatoração com exemplos Antes/Depois e padrão Arrange, Act, Assert (`userService.clean.test.js`)
3. Configuração e execução do ESLint com `eslint-plugin-jest`, limitações e conclusão

Outubro de 2026

Relatório técnico · 4 páginas

1 Análise dos Test Smells

A suíte `test/userService.smelly.test.js` passa por completo (4 testes aprovados e 1 ignorado), mas esconde problemas de projeto. Os três smells abaixo foram verificados diretamente no código do repositório original; os números de linha correspondem ao arquivo.

1.1 Lógica condicional no teste Conditional Test Logic

Caracterização O teste usa laços ou desvios (`for`, `if/else`) para decidir o que executar e o que verificar. Um teste deveria seguir sempre o mesmo caminho e checar um único cenário.

USERSERVICE.SMELLY.TEST.JS · "DEVE DESATIVAR USUÁRIOS SE ELES NÃO FOREM ADMINISTRADORES"

```
40 for (const user of todosOsUsuarios) {
41   const resultado = userService.deactivateUser(user.id);
42   if (!user.isAdmin) {
43     // Este expect só roda para o usuário comum.
44     expect(resultado).toBe(true);
45     const usuarioAtualizado = userService.getUserById(user.id);
46     expect(usuarioAtualizado.status).toBe('inativo');
47   } else {
48     // E este só roda para o admin.
49     expect(resultado).toBe(false);
50   }
51 }
```

Por que é problema Dois comportamentos distintos (usuário comum é desativado; administrador não é) ficam no mesmo teste, e cada `expect` só roda em um ramo. Se uma falha ocorrer, o nome do teste não indica qual regra quebrou. Além disso, se a lista mudar (por exemplo, só administradores), o ramo do usuário comum deixa de ser verificado sem que nada avise. Também falta conferir que o status do administrador continuou `'ativo'`.

1.2 Exceção verificada com try/catch Exception Handling

Caracterização O teste trata a exceção manualmente em vez de usar a asserção própria do framework. A verificação só acontece se o `catch` for alcançado.

USERSERVICE.SMELLY.TEST.JS · "DEVE FALHAR AO CRIAR USUÁRIO MENOR DE IDADE"

```
70 try {
71   userService.createUser('Menor', 'menor@email.com', 17);
72 } catch (e) {
73   expect(e.message).toBe('O usuário deve ser maior de idade.');
74 }
```

Por que é problema Se `createUser` deixar de lançar o erro, nenhum `expect` é executado e o teste passa, gerando um falso positivo. Isso foi confirmado por experimento: em uma cópia do projeto, `if (idade < 18)` foi trocado por `if (false)` em `src/userService.js` e o Jest foi executado. A suíte original continuou com PASS; a suíte refatorada acusou FAIL com "Received function did not throw".

1.3 Teste desabilitado e vazio Ignored / Empty Test

Caracterização O teste é marcado com `test.skip` e não possui corpo nem asserções, apenas um `TODO`.

USERSERVICE.SMELLY.TEST.JS

```
77 test.skip('deve retornar uma lista vazia quando não há usuários', () => {
78   // TODO: Implementar este teste depois.
79 });
```

Por que é problema O relatório do Jest mostra "1 skipped", mas a suíte continua verde, o que dá falsa sensação de cobertura. O cenário de base vazia de `generateUserReport()` nunca é exercitado e o `TODO` tende a ser esquecido. O nome também descreve "lista vazia", enquanto o método real devolve um texto.

Outros problemas observados (fora do escopo dos três smells): o primeiro teste (linhas 18–31) é um *Eager Test*, com "Act 1" e "Act 2" verificando criação e busca juntas. O teste do relatório (linhas 61–63) compara a linha formatada completa e o cabeçalho exato, acoplando-se à apresentação, como o próprio comentário da linha 60 admite.

2 Refatoração dos testes

Os testes limpos estão em `test/userService.clean.test.js`, sem alteração no código da aplicação. Cada teste cobre um comportamento e segue o padrão **Arrange** (preparar), **Act** (executar) e **Assert** (verificar).

2.1 Lógica condicional → um teste por regra de negócio

ANTES · SMELLY, LINHAS 40–51

```
40  for (const user of todosOsUsuarios) {
41    const resultado = userService.deactivateUser(user.id);
42    if (!user.isAdmin) {
43      // Este expect só roda para o usuário comum.
44      expect(resultado).toBe(true);
45      const usuarioAtualizado = userService.getUserById(user.id);
46      expect(usuarioAtualizado.status).toBe('inativo');
47    } else {
48      // E este só roda para o admin.
49      expect(resultado).toBe(false);
50    }
51  }
```

DEPOIS · CLEAN, LINHAS 91–101

```
91  test('não desativa um administrador', () => {
92    // Arrange
93    const administrador = userService.createUser('Admin', 'admin@email.com', 40, true);
94
95    // Act
96    const resultado = userService.deactivateUser(administrador.id);
97
98    // Assert
99    expect(resultado).toBe(false);
100   expect(userService.getUserById(administrador.id).status).toBe('ativo');
101 });
```

O laço virou três testes: *desativa um usuário comum* (linhas 79–89), *não desativa um administrador* (acima) e *retorna false quando o usuário não existe*, antes não coberto. Toda asserção executa sempre, e o status do administrador passou a ser conferido.

2.2 try/catch → asserção explícita de exceção

ANTES · SMELLY, LINHAS 66–75

```
66  test('deve falhar ao criar usuário menor de idade', () => {
 ⋮  // (trecho omitido)
70    try {
71      userService.createUser('Menor', 'menor@email.com', 17);
72    } catch (e) {
73      expect(e.message).toBe('O usuário deve ser maior de idade.');
74    }
75  });
```

DEPOIS · CLEAN, LINHAS 31–38

```
31  test('rejeita usuário menor de idade', () => {
32    // Arrange
33    const idadeMinimaInvalida = 17;
34
35    // Act / Assert
36    expect(() => userService.createUser('Menor', 'menor@email.com', idadeMinimaInvalida))
37      .toThrow('O usuário deve ser maior de idade.');
38  });
```

`toThrow` falha se a função **não** lançar o erro, eliminando o falso positivo; o padrão também cobre os campos obrigatórios.

2.3 Teste ignorado → cenário implementado

DEPOIS · CLEAN, LINHAS 116–125

```
116 test('informa que não há usuários quando a base está vazia', () => {
 ⋮  // (trecho omitido)
120   // Act
121   const relatorio = userService.generateUserReport();
122
123   // Assert
124   expect(relatorio).toContain('Nenhum usuário cadastrado.');
125 });
```

O `test.skip` vazio (linhas 77–79, seção 1.3) virou um teste ativo, com nome ajustado ao retorno real (texto).

Melhorias obtidas

Aspecto	Suíte original	Suíte refatorada
Execução no Jest	4 aprovados, 1 ignorado	12 aprovados, nenhum ignorado
Validação de idade removida	Continua passando (falso positivo)	Falha e aponta o teste exato
Acoplamento ao formato	Linha completa e cabeçalho exato	Apenas `'Nome: Alice'` e `'Status: ativo'`

3 Análise estática com ESLint e conclusão

.ESLINTRC.JSON

```
{
  "env": {
    "es2021": true,
    "node": true,
    "jest/globals": true
  },
  "extends": [
    "eslint:recommended",
    "plugin:jest/recommended"
  ],
  "parserOptions": {
    "ecmaVersion": "latest",
    "sourceType": "script"
  },
  "plugins": ["jest"],
  "ignorePatterns": ["node_modules/"],
  "rules": {
    "jest/no-disabled-tests": "warn",
    "jest/no-conditional-expect": "error",
    "jest/no-identical-title": "error"
  }
```

O ESLint 8.57.1 foi configurado com `eslint-plugin-jest` 28.14.0. O preset `plugin:jest/recommended` já inclui as regras abaixo; elas foram declaradas explicitamente para deixar claros o nível e a intenção:

- `jest/no-conditional-expect` (erro): acusa `expect` dentro de `if`, `catch` ou callbacks condicionais. Cobre os smells 1.1 e 1.2.
- `jest/no-disabled-tests` (aviso): acusa `test.skip`, `xit` e similares. Cobre o smell 1.3.
- `jest/no-identical-title` (erro): impede títulos duplicados, que confundem relatórios.

Também foram adicionados ao `package.json` o script `"lint": "eslint ."` e as dependências `eslint` e `eslint-plugin-jest`. Ambiente da execução: Node 22.22.0, Jest 29.7.0.

Execução real

terminal — /home/claude/work (test-smelly)

```
$ npx eslint test/userService.smelly.test.js

/home/claude/work/test/userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)

$ echo $?
1
$ npx eslint test/userService.clean.test.js
$ echo $?
0
```

Figura 1 — Saída real do ESLint, executado em 18/10/2026 (Node 22.22.0) no ambiente de nuvem usado para preparar este relatório; o texto foi capturado do terminal e renderizado como imagem, sem edição. Código de saída 1 para o arquivo original e 0 para o refatorado.

O que o ESLint detectou e suas limitações

No arquivo original foram 6 problemas. Três `no-conditional-expect` nas linhas 44, 46 e 49 (ramos do `if/else` do smell 1.1), um na linha 73 (`expect` dentro do `catch`, smell 1.2), além de `no-disabled-tests` e `expect-expect` na linha 77 (smell 1.3). O arquivo refatorado não gerou nenhum aviso, e o Jest aprovou os 12 testes da nova suíte.

A ferramenta, porém, só reconhece padrões sintáticos. Não apontou o *Eager Test* nem o acoplamento ao formato do relatório, que dependem da intenção do teste, e não executa código: o falso positivo do smell 1.2 só foi comprovado rodando o Jest com a validação removida. Ausência de avisos não garante asserções relevantes nem boa cobertura.

Conclusão

A suíte original passava, mas isso não significava que protegia o sistema: um teste podia passar com a regra de idade removida, outro misturava cenários em ramos condicionais e um terceiro nem era executado. Ao refatorar com um comportamento por teste, nomes descritivos e o padrão AAA, a suíte ficou mais fácil de ler e de diagnosticar e passou a falhar quando deve. O ESLint com `eslint-plugin-jest` é útil como barreira automática, pois impede que esses padrões voltem ao código. Ainda assim, precisa ser combinado com revisão humana e com a execução dos testes, já que parte dos smells só aparece quando se analisa o que o teste deveria garantir.